

SENIOR ENGINEER INTERVIEW PREP

Gokulakonda Gautham

DOCUMENT 10 OF 10

CI/CD, Production Debugging & Observability

10 Questions | Full CI/CD Pipeline | Secrets | Incident Response | Profiling | HA | LLM/RAG Production

CI/CD, Production Debugging & Observability

Q1: Walk through a complete CI/CD pipeline — from git push to production.

A mature CI/CD pipeline is the difference between deploying once a week with anxiety and deploying 20 times a day with confidence. Each stage is a quality gate — catch bugs as early and cheaply as possible.

Stage 1 — Pre-commit (local): git hooks (pre-commit) run: lint (ruff, eslint), type-check (mypy, tsc), auto-format (black, prettier). Fails fast before the code even leaves the developer's machine. Fast: < 10 seconds.

Stage 2 — CI on pull request: Triggered on every PR. Runs: unit tests, integration tests, security scans (Trivy for Docker, Snyk for dependencies), code coverage check (fail if < 80%), SAST (static analysis). Reviewer cannot merge until all checks pass.

Stage 3 — Build and publish: On merge to main: build Docker image, tag with git SHA (not 'latest'), push to container registry (GCR, ECR, GHCR). Image SHA is the immutable deployment artifact — what you deploy is exactly what passed CI.

Stage 4 — Deploy to staging: Automatically deploy to staging environment. Run smoke tests (critical path end-to-end). Run performance tests (k6, Locust) — fail if p99 latency regresses > 10%. Staging mirrors production config, smaller scale.

Stage 5 — Deploy to production: Manual approval gate (or auto for trusted teams). Rolling deploy (K8s) or canary. Monitor error rate and latency for 10 minutes. Automated rollback if metrics degrade. Notify team on Slack: deployed, sha, author, changelog.

```
# GitHub Actions – complete pipeline
name: CI/CD
on:
  pull_request: { branches: [main] }
  push: { branches: [main] }

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres: { image: postgres:16, env: { POSTGRES_PASSWORD: test } }
      redis: { image: redis:7 }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.12' }
      - run: pip install -r requirements.txt
      - run: ruff check . # lint
      - run: mypy app/ # type check
      - run: pytest --cov=app --cov-fail-under=80 # test + coverage

  build:
    needs: test
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: actions/checkout@v4
      - name: Build and push image
        run: |
          docker build -t gcr.io/my-project/api:${{ github.sha }} .
          docker push gcr.io/my-project/api:${{ github.sha }}

  deploy-staging:
    needs: build
    steps:
      - run: |
          kubectl set image deployment/api \
            api=gcr.io/my-project/api:${{ github.sha }} \
```

```
--namespace staging
kubectl rollout status deployment/api --namespace staging

deploy-prod:
  needs: deploy-staging
  environment: production # requires manual approval
  steps:
    - run: |
        kubectl set image deployment/api \
          api=gcr.io/my-project/api:${{ github.sha }} \
          --namespace production
```

Q2: Git branching strategies — trunk-based vs gitflow, and why it matters.

Your branching strategy determines how frequently you integrate code, how long feature branches live, and how painful merges are. It's a team decision that directly affects deployment frequency and cycle time.

Gitflow: Long-lived branches: main, develop, feature/*, release/*, hotfix/*. Features branch from develop, merge back to develop. Release branch for QA. Hotfixes from main. Complex, many merge points, large integration diffs. Suited for software with scheduled releases (mobile apps, on-prem software).

Trunk-based development (TBD): Everyone integrates to main (trunk) at least once a day. No long-lived feature branches. Short-lived branches (1-2 days max) or direct commits to trunk. Feature flags hide incomplete work. Forces small, incremental changes. Enables continuous deployment. Used by Google, Facebook, Netflix at scale.

Why TBD is better for most teams: Long-lived branches accumulate drift — the longer a branch lives, the bigger and more painful the merge. TBD forces integration early, when conflicts are small. It also forces you to deploy incomplete features behind flags, which is actually safer — you can disable with a flag instead of rolling back a deployment.

```
# Trunk-based development workflow
git checkout -b feat/add-search-filter # short-lived feature branch
# ... work for 1-2 days max ...
git push origin feat/add-search-filter
# Open PR -> CI runs -> code review -> merge
# Branch deleted after merge

# Feature flag hides incomplete work
if feature_flags.is_enabled('new-search-filter', user_id):
    return new_search_handler(query)
else:
    return legacy_search_handler(query)

# Commit message convention (Conventional Commits)
feat: add pagination to user list endpoint
fix: resolve race condition in payment processor
perf: add composite index on orders(user_id, created_at)
docs: update API documentation for v2 endpoints
chore: upgrade FastAPI to 0.111.0
# These enable: automated changelogs, semantic versioning, release notes
```

Q3: Secrets management — how to handle credentials in a CI/CD pipeline and production.

Secrets leaked in code, logs, or environment variables are one of the most common security incidents. A principled approach to secrets management eliminates entire classes of vulnerabilities.

Never in code or git: No secrets in source files, Dockerfiles, docker-compose.yml, or K8s manifests committed to git. git-secrets and gitleaks scan for accidentally committed secrets. Secret scanners run in CI — block the merge if a secret pattern is detected.

Kubernetes Secrets: Store as base64-encoded K8s Secret objects (not plain ConfigMaps). Mount as environment variables or volume files. Encrypt at rest using a KMS provider (AWS KMS, GCP KMS). Never log environment variables — they contain the decoded secrets.

External secret managers: Vault (HashiCorp), AWS Secrets Manager, GCP Secret Manager. Secrets live in the vault, not in K8s or env files. External Secrets Operator syncs secrets from vault to K8s Secrets automatically. Rotation is built-in — no service restart needed.

```
# K8s Secret (stored encrypted at rest with KMS)
kubectl create secret generic api-secrets \
  --from-literal=database_url='postgresql://...' \
  --from-literal=secret_key='...' \
  --namespace production

# Reference in Deployment (never hardcode values)
env:
- name: DATABASE_URL
  valueFrom:
    secretKeyRef:
      name: api-secrets
      key: database_url

# External Secrets Operator — sync from AWS Secrets Manager
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata: { name: api-secrets }
spec:
  refreshInterval: 1h
  secretStoreRef: { name: aws-secrets-manager, kind: ClusterSecretStore }
  target: { name: api-secrets }
  data:
  - secretKey: database_url
    remoteRef: { key: 'prod/api/db', property: url }

# GitHub Actions — use encrypted secrets, not env vars in code
env:
  DATABASE_URL: ${ secrets.DATABASE_URL } # stored in GitHub Secrets UI
```

Rotation: Rotate secrets regularly — especially after team member offboarding. Automated rotation in AWS Secrets Manager calls a Lambda that updates the DB password and the secret value simultaneously. Zero-downtime rotation: update secret, old connections drain, new connections use new secret.

Q4: How do you debug a production incident? Walk through your process.

Production debugging under pressure is a senior engineering skill. The process matters as much as the technical knowledge — panic leads to thrashing. A systematic approach finds the root cause faster.

Step 1 — Triage: define the blast radius: How many users are affected? Which features? Is it total outage or partial? What changed recently (deployment, config change, traffic spike)? Get this in Slack immediately so the team has context. Don't start fixing until you understand the scope.

Step 2 — Stabilize first, fix later: If you have a rollback, use it. Restoring service is higher priority than understanding the root cause. Roll back the last deployment. Increase DB connection pool. Add capacity. Get the site up, then investigate why.

Step 3 — Find the signal: Error rate spike in Datadog/Grafana — when did it start, which service? Correlate with deployment timeline. Check logs for the first error — what's the stack trace? Check distributed traces — which service is slow or erroring? Check DB slow query log, Redis memory, Kafka consumer lag.

Step 4 — Isolate and fix: Reproduce in staging if possible. Minimal change to fix the immediate issue. Do NOT refactor during an incident — smallest possible change. Deploy fix, monitor for 10 minutes, confirm recovery.

Step 5 — Post-mortem (blameless): 24-48 hours after the incident: timeline of events, root cause, contributing factors, what went well, what to improve, action items with owners and deadlines. Blameless means the system failed — not a person. Psychological safety is required for honest post-mortems.

```
# Debugging toolkit – commands you need immediately

# K8s: check pod status and recent events
kubectl get pods -n production --sort-by=.status.startTime
kubectl describe pod <pod-name> -n production # events, resource limits
kubectl logs <pod-name> -n production --previous # crashed pod logs
kubectl logs <pod-name> -n production --since=10m # recent logs

# Resource usage
kubectl top pods -n production
kubectl top nodes

# DB: find slow queries right now
SELECT pid, now() - query_start AS duration, query, state
FROM pg_stat_activity
WHERE state != 'idle' AND query_start < now() - interval '1 second'
ORDER BY duration DESC;

# DB: blocking locks
SELECT blocked.pid, blocked.query, blocking.pid AS blocking_pid
FROM pg_stat_activity blocked
JOIN pg_stat_activity blocking ON blocking.pid = ANY(pg_blocking_pids(blocked.pid))
WHERE blocked.cardinality(pg_blocking_pids(blocked.pid)) > 0;

# Redis: check memory and slow commands
redis-cli INFO memory | grep used_memory_human
redis-cli SLOWLOG GET 10 # last 10 slow commands
redis-cli MONITOR # real-time command stream (dev only!)

# Check what changed – git log since last deploy
git log --oneline v1.2.3..v1.2.4
```

Q5: Observability — metrics, logs, traces: the three pillars and how they work together.

Observability is the ability to understand the internal state of a system from its external outputs. The three pillars — metrics, logs, traces — answer different questions and work best together.

Metrics — what is happening?: Numeric time-series data aggregated over time. Fast to query, cheap to store. Answer: is the error rate up? What's the p99 latency? How many requests/sec? Tools: Prometheus (collection + storage), Grafana (visualization), AlertManager (alerting).

Logs — what happened?: Structured event records with full context. Answer: what exactly happened for this specific request? What was the error message and stack trace? Tools: ELK (Elasticsearch, Logstash, Kibana), Grafana Loki, Datadog Logs, CloudWatch.

Traces — why did it happen?: End-to-end request journey across all services. Answer: which service added 200ms to this request? Where did the error originate in the call chain? Tools: Jaeger, Zipkin, Tempo, Datadog APM.

```
# Prometheus metrics in FastAPI
from prometheus_client import Counter, Histogram, generate_latest
import time

REQUEST_COUNT = Counter(
    'http_requests_total',
```

```

        'Total HTTP requests',
        ['method', 'endpoint', 'status']
    )
    REQUEST_LATENCY = Histogram(
        'http_request_duration_seconds',
        'HTTP request latency',
        ['method', 'endpoint'],
        buckets=[.005, .01, .025, .05, .1, .25, .5, 1, 2.5]
    )

    @app.middleware('http')
    async def metrics_middleware(request: Request, call_next):
        start = time.time()
        response = await call_next(request)
        duration = time.time() - start
        REQUEST_COUNT.labels(request.method, request.url.path, response.status_code).inc()
        REQUEST_LATENCY.labels(request.method, request.url.path).observe(duration)
        return response

    @app.get('/metrics')
    async def metrics(): return Response(generate_latest(), media_type='text/plain')

    # Grafana dashboard – key panels for a service
    # 1. Request rate: rate(http_requests_total[5m])
    # 2. Error rate: rate(http_requests_total{status=~'5..'}[5m])
    # 3. Latency p50/p95/p99: histogram_quantile(0.99, ...)
    # 4. Active goroutines / connections
    # 5. DB connection pool usage

```

SLO and alerting: Define SLOs (Service Level Objectives): 99.9% of requests complete in < 200ms, error rate < 0.1%. Alert when SLO burn rate is too high — not on individual metric spikes. Burn rate alerting: if you're consuming your error budget 14x faster than normal, page immediately. This avoids alert fatigue from noisy threshold alerts.

USE and RED methods: USE (for infrastructure): Utilization, Saturation, Errors. RED (for services): Rate, Errors, Duration. Start with RED for every API endpoint — Rate (requests/sec), Errors (error rate), Duration (latency percentiles). These three metrics tell you almost everything about a service's health.

Q6: How do you handle database migrations safely in production?

Database migrations are the most dangerous part of a deployment. A bad migration can take down production in seconds — and unlike code, you can't roll back data changes easily.

Expand/contract pattern: The safest approach for zero-downtime migrations. Instead of changing schema in one step, do it in three deployments: (1) Expand — add new column/table alongside old (both coexist). (2) Migrate — backfill data, dual-write. (3) Contract — remove old column/table after all traffic uses new.

Never do in one migration: Rename a column (drops and recreates — existing queries break). Add a NOT NULL column without a default (fails for existing rows). Drop a column that code still reads. Change a column type that requires rewriting all rows. Each of these needs expand/contract.

```

# DANGEROUS – adding NOT NULL column breaks existing rows
ALTER TABLE users ADD COLUMN phone VARCHAR(20) NOT NULL;
# ERROR: column 'phone' of relation 'users' contains null values

# SAFE – add nullable first, backfill, then add constraint
-- Migration 1 (deploy with current code):
ALTER TABLE users ADD COLUMN phone VARCHAR(20);

-- Migration 2 (background job – backfill in batches):
UPDATE users SET phone = '' WHERE phone IS NULL
AND id BETWEEN $start AND $end; -- batch to avoid table lock

```

```
-- Migration 3 (after all rows populated):
ALTER TABLE users ALTER COLUMN phone SET NOT NULL;

# Alembic (Python) migration with safety check
def upgrade():
    # 1. Add column nullable
    op.add_column('users', sa.Column('phone', sa.String(20)))

def downgrade():
    op.drop_column('users', 'phone')

# For large tables – avoid full table lock
# PostgreSQL: CREATE INDEX CONCURRENTLY (doesn't lock table)
CREATE INDEX CONCURRENTLY idx_users_phone ON users(phone);
# vs blocking:
CREATE INDEX idx_users_phone ON users(phone); # locks table!
```

Migration in CI/CD: Run migrations before deploying new code (not after). If migration fails, the old code still works. If new code deploys before migration, new code fails against old schema. Alembic check in CI — verify no unapplied migrations on main. Lock migration table to prevent concurrent migrations across multiple deploying instances.

Large table migrations: For tables with millions of rows: use `pg_repack` to rebuild tables without locks. Batch updates (`UPDATE ... WHERE id BETWEEN X AND Y LIMIT 1000`) to avoid long-running transactions. Use `pt-online-schema-change` (Percona) for MySQL. These tools replicate changes incrementally, swap tables atomically at the end.

Q7: How do you approach performance profiling in production Python and Go services?

Profiling in production is different from profiling in development. You need low-overhead continuous profiling that doesn't affect user traffic, and you need to correlate profiling data with request traces.

Python — cProfile and py-spy: `cProfile`: built-in, adds overhead, better for controlled benchmarks. `py-spy`: sampling profiler that attaches to a running Python process without modifying code. Zero overhead in production. Generates flamegraphs showing where CPU time is spent.

```
# py-spy – attach to live production process
py-spy record --pid 12345 --output profile.svg --duration 30
# Opens as SVG flamegraph – wider bars = more CPU time

# py-spy top – live CPU usage per function (like htop for Python)
py-spy top --pid 12345

# Austin – frame pointer based, even lower overhead
austin -p 12345 -o profile.austin

# FastAPI endpoint-level timing with middleware
import cProfile, pstats, io

@app.middleware('http')
async def profile_middleware(request: Request, call_next):
    if request.headers.get('X-Profile') == settings.profile_token:
        pr = cProfile.Profile()
        pr.enable()
        response = await call_next(request)
        pr.disable()
        s = io.StringIO()
        pstats.Stats(pr, stream=s).sort_stats('cumulative').print_stats(20)
        print(s.getvalue()) # log to stdout
        return response
    return await call_next(request)
```

```
# Memory profiling – find leaks
from memray import Tracker
with Tracker('output.bin'):
    run_suspected_leaky_code()
# memray flamegraph output.bin -- visualize allocations
```

Go — pprof: Go has built-in profiling via net/http/pprof. Import it, and /debug/pprof/ endpoints become available. Zero configuration. CPU profile, heap profile, goroutine dump, mutex contention — all available on-demand.

```
// Go – enable pprof (in your server setup)
import _ 'net/http/pprof' // side-effect import registers handlers

// Start debug server on separate port (never expose to public)
go func() {
    log.Println(http.ListenAndServe('localhost:6060', nil))
}()

// Collect CPU profile for 30 seconds
go tool pprof http://localhost:6060/debug/pprof/profile?seconds=30

// Heap profile – find memory leaks
go tool pprof http://localhost:6060/debug/pprof/heap

// Goroutine dump – find goroutine leaks
curl http://localhost:6060/debug/pprof/goroutine?debug=2

// In pprof interactive shell:
(pprof) top10           // top CPU consumers
(pprof) web             // open flamegraph in browser
(pprof) list myFunc     // annotated source with CPU cost per line

// Continuous profiling in production: Pyroscope
// Attaches to Go process, sends profiles to Pyroscope server
// Correlates CPU profiles with request traces via labels
pyroscope.Start(pyroscope.Config{
    ApplicationName: 'user-service',
    ServerAddress:   'http://pyroscope:4040',
    ProfileTypes:    []pyroscope.ProfileType{pyroscope.ProfileCPU,
pyroscope.ProfileMemAlloc},
})
```

Continuous profiling: Pyroscope, Grafana Phlare, or Google Cloud Profiler run continuously in production at < 1% overhead. They correlate profiles with deployments — you can literally see which commit caused a CPU regression. This is the future of production profiling.

Q8: How do you design for high availability — what does 99.9% vs 99.99% uptime actually require?

99.9% uptime = 8.7 hours downtime/year. 99.99% = 52 minutes/year. 99.999% = 5 minutes/year. Each 9 is an order of magnitude harder and more expensive. Understanding what's actually required to achieve each level is what separates architects from developers.

99.9% (three nines): Single-region. Redundant instances behind a load balancer. Health checks remove failed instances. DB primary with a hot standby. Automated failover (Patroni). RTO: minutes. This is achievable with a small team and standard cloud infrastructure.

99.99% (four nines): Multi-AZ (availability zone) deployment within a region. All critical components are AZ-redundant: DB, Redis, Kafka, app servers. Network partition between AZs: traffic shifts to healthy AZs automatically. DB: synchronous multi-AZ replication (Aurora, RDS Multi-AZ). RTO: < 1 minute.

99.999% (five nines): Multi-region active-active or active-passive. Global load balancing (Route53, Cloudflare). Data replication across regions (async — adds latency, or sync — adds write latency). Region-level failures (rare

but happen: us-east-1 outages). Requires conflict resolution strategy for multi-region writes. Expensive: 5-10x infrastructure cost.

```
# Availability calculation
# If each component has 99.9% uptime:
# Serial dependencies: A -> B -> C
# System uptime = 0.999 * 0.999 * 0.999 = 0.997 = 99.7%
# Adding services REDUCES overall availability

# Parallel redundancy improves availability:
# Two load-balanced instances each at 99%:
# P(both fail) = 0.01 * 0.01 = 0.0001 = 99.99% system uptime

# SLA, SLO, SLI – the hierarchy
# SLI (Service Level Indicator): measured metric
#   e.g., 'request success rate over 5 min window'
# SLO (Service Level Objective): internal target
#   e.g., 'SLI must be >= 99.9%'
# SLA (Service Level Agreement): external contractual commitment
#   e.g., 'we guarantee 99.5% uptime or you get a credit'
# SLA is always lower than SLO – you need headroom

# Error budget: 99.9% SLO = 0.1% error budget
# = 43.8 minutes/month of allowed downtime
# Spent on: incidents, planned maintenance, risky deploys
# Budget exhausted = freeze risky changes until next month
```

Chaos engineering: Netflix's approach: deliberately inject failures in production (kill random pods, simulate network latency, drop DB connections) to verify the system recovers automatically. Chaos Monkey, LitmusChaos, AWS Fault Injection Simulator. You can't trust your HA design until you've deliberately broken it and watched it recover.

Q9: Architecture tradeoffs — how to communicate them clearly in interviews.

Senior engineers are distinguished not by knowing the 'right' answer but by understanding the tradeoffs of every option and choosing based on context. Interviewers want to see this reasoning process, not just the conclusion.

The tradeoff framework: For every architectural decision, articulate: (1) What problem does this solve? (2) What does it cost (complexity, latency, consistency, money)? (3) Under what conditions does the alternative win? (4) What is your recommendation given the stated constraints?

SQL vs NoSQL: SQL: ACID, joins, flexible queries, schema enforced, proven tooling. Best for: relational data, financial records, complex queries. NoSQL (document): schema-flexible, horizontal write scaling, denormalized for read patterns. Best for: user profiles, product catalogs, variable-schema data. NoSQL (wide-column): extreme write throughput, time-series. Cassandra for IoT, clickstream, telemetry.

Sync vs async communication: Sync REST: simpler, immediate feedback, tight coupling, failure propagates. Async Kafka: decoupled, buffered, complex, eventual consistency. Rule: if the response is needed to proceed, sync. If you can proceed without waiting for the downstream result, async.

Normalize vs denormalize: Normalize: consistency, less storage, update in one place, slower reads (joins). Denormalize: faster reads, data duplication, update in multiple places, risk of inconsistency. Normalize the write path; denormalize the read path via materialized views or read replicas.

Strong vs eventual consistency: Strong: always correct, higher latency, lower availability. Eventual: lower latency, higher availability, temporary inconsistency. Use strong for: money, inventory, seat reservations. Use eventual for: social feeds, read counts, recommendation scores.

```
# How to answer a tradeoff question in an interview
```

```
Interviewer: 'Should we use PostgreSQL or Cassandra for storing user sessions?'
```

```
Good answer structure:
```

```
1. Clarify: 'How many users? Read/write ratio? Global distribution needed?'
```

2. Compare on relevant dimensions:
PostgreSQL: ACID, supports complex queries on session data, familiar tooling, single-region scales to ~50k writes/sec with pgBouncer.
Cassandra: multi-region active-active, linear horizontal scaling, eventual consistency, simpler data model (key-value).
3. Give a recommendation with conditions:
'For < 10M users, single-region: PostgreSQL is simpler and sufficient.
For > 100M users or global distribution: Cassandra or DynamoDB because sessions are key-value access patterns and global low-latency reads are critical. I would start with Redis as the primary session store (it's just a TTL'd key-value) and PostgreSQL as the fallback, which covers 99% of cases at a fraction of the complexity.'
4. Identify the risk in your recommendation:
'The risk with Redis-only is data loss on restart without persistence. We mitigate with AOF and Redis Sentinel for HA.'

Q10: LLM integration, RAG pipelines, and agentic workflows — production considerations.

Your resume lists LangGraph, LangChain, RAG, FAISS, Chroma, and MCP. These are increasingly common in senior engineering interviews at AI-forward companies. Know the production gotchas, not just the happy path.

RAG (Retrieval-Augmented Generation): Instead of asking the LLM to remember everything, retrieve relevant context from a vector database and inject it into the prompt. Pipeline: ingest documents → chunk → embed → store in vector DB. Query: embed query → similarity search → retrieve top-K chunks → build prompt → call LLM → return answer.

```
# RAG pipeline – production considerations

# 1. Chunking strategy matters
# Fixed-size chunks: simple, may split mid-sentence
# Semantic chunks: split on paragraph/section boundaries – better retrieval
# Chunk size: 256-512 tokens is common. Too small = no context.
#                                     Too large = diluted relevance.

# 2. Embedding model consistency
# You MUST use the same embedding model for indexing AND querying
# Switching models requires re-embedding the entire corpus
EMBEDDING_MODEL = 'text-embedding-3-small' # pin this version

# 3. Retrieval – beyond simple cosine similarity
# Hybrid search: combine dense (vector) + sparse (BM25 keyword) retrieval
# Re-ranking: use a cross-encoder to re-rank top-K results
# Maximal Marginal Relevance: avoid returning near-duplicate chunks

# 4. FAISS vs Chroma vs pgvector
# FAISS: fastest, in-memory, no persistence natively, exact or approximate
# Chroma: local dev, small-medium corpora, built-in embedding
# pgvector: production, stays in Postgres, ACID, exact similarity search
# Pinecone/Weaviate: managed, horizontal scale, production-grade

# 5. Evaluating RAG quality
# Retrieval metrics: recall@K (did the relevant doc appear in top-K?)
# Generation metrics: faithfulness (answer grounded in retrieved context?)
# RAGAS framework: automates RAG evaluation
```

Agentic workflows (LangGraph): Agents use LLMs to decide which tools to call in sequence. LangGraph models this as a state machine — nodes are actions/LLM calls, edges define transitions. Key considerations: loop detection (agents can get stuck), max iterations, tool timeout, state serialization for resumability.

Production LLM concerns: Latency: LLM calls are 1-30 seconds. Use streaming (SSE) for perceived responsiveness. Cost: every token costs money — cache common prompts (semantic caching with Redis + embeddings). Rate limits: implement exponential backoff and circuit breakers around LLM provider APIs. Prompt injection: user input can manipulate system prompts — sanitize and validate.

MCP (Model Context Protocol): Anthropic's open standard for connecting LLMs to tools and data sources. An MCP server exposes tools (functions), resources (data), and prompts. The LLM client discovers available tools and calls them. Your resume mentions this — be ready to explain: it standardizes the tool-calling interface so any MCP-compatible LLM can use any MCP server without custom integration code.
